CYBER SECURITY

Practical Web Application Security Assessment

Assignment 1

Course	**Cyber Security**
Program	**BS Computer Science**
Section	**E4**
Instructor	**Hafiz Usama Ishtiaq**
Assignment Title	**Practical Web Application Security Assessment Using Altoro Mutual**
Target Application	**Altoro Mutual (demo banking site) — https://demo.testfire.net/**
Total Marks	**100**
Submission Format	**PDF report with screenshots**
Due Date	**28-10-2026**

Scope and Authorization — Read Before You Begin

Altoro Mutual (demo.testfire.net) is a publicly available, intentionally vulnerable demonstration banking application maintained specifically for security-testing training and education. No real customers, real money, or real data exist on this site.

You are authorized to perform the specific, non-destructive tests described in this guide against demo.testfire.net only, or against the named alternative training lab (PortSwigger Web Security Academy) where this guide directs you to it.

You are NOT authorized to test any other website, application, or system, including real banks or any site you use personally, using the techniques in this assignment. Doing so is both illegal and a serious academic integrity violation, regardless of intent.

All testing must be non-destructive. Do not attempt denial-of-service testing, credential theft, brute-forcing, unauthorized data extraction, or any action beyond what this guide instructs.

Use only fictional test data (made-up names, emails, and account details), never your own real personal or financial information.

Instructor: Hafiz Usama Ishtiaq
Faculty of Information Technology
University of Central Punjab
Lahore, Pakistan

Part A Assignment Questions

Answer every question below. Questions marked with a procedure reference should be completed using the step-by-step instructions in Part B of this guide before you write your answer.

Task 1: Website Exploration

1. Browse the Altoro Mutual homepage and navigate through at least three of its main sections (for example: Personal, Small Business, Inside Altoro Mutual, and the Online Banking sign-in area). Identify and briefly describe five important features or functional elements of the website.
2. Identify the login (Sign In), Search, and Feedback functionalities and state where each one is located on the site.
3. For each of these three functionalities, explain in two to three sentences why that type of feature commonly needs security testing.

Task 2: SQL Injection Investigation

1. In your own words, explain what SQL injection is at a conceptual level.
2. Describe, in general terms, the kind of coding mistake that can make a web application vulnerable to SQL injection.
3. Identify the functionality on Altoro Mutual that is the appropriate, instructor-approved target for this investigation, and justify your choice.
4. Following the procedure in Part B, carry out the approved, non-destructive test and record exactly what you entered and what the application returned.
5. Based on your own observation, explain what result would indicate unexpected input handling versus normal, properly-handled input.
6. Explain the possible impact of a real SQL injection vulnerability on the confidentiality and integrity of banking data.
7. Recommend at least two concrete prevention techniques.

Task 3: Cross-Site Scripting (XSS) Investigation

1. Define reflected XSS and stored XSS, and explain the difference between them.
2. Identify a suitable input field on Altoro Mutual for examining how user-supplied input is displayed back to a user, and justify your choice.
3. Following the procedure in Part B, carry out a harmless test in the authorized environment and record exactly what you entered and what was displayed in response.
4. Based on what you observed, explain how to interpret the result — was your input treated as plain text, or interpreted as active content by the browser?
5. Explain the potential impact of an XSS vulnerability on the users of a banking website.
6. Recommend at least two concrete prevention techniques.

Task 4: Security Analysis

Answer the following in your own words, using correct technical terminology:

1. Differentiate between SQL injection and XSS.
2. Which parts of the CIA triad (Confidentiality, Integrity, Availability) may be affected by SQL injection, and how?
3. Why is server-side authorization important in banking applications, even when client-side checks already exist?
4. Explain how parameterized queries help prevent SQL injection.
5. Explain how context-appropriate output encoding helps prevent XSS.

Task 5: Report and Recommendations

Compile a professional security assessment report containing the following sections. Guidance for each is given in Part B.5 of this guide.

- Introduction and objectives
- Website overview
- Testing methodology
- Tools used
- Screenshots and observations
- Findings and their potential impact
- Recommended security fixes
- Conclusion

Part B : Step-by-Step Practical Instruction Guide

Before You Start Testing

This assignment teaches you to recognize indicators of common web application vulnerabilities through careful, controlled observation — not to "hack" anything. Every test in this guide is deliberately simple, harmless, and reversible.

demo.testfire.net is a public, third-party website that Anthropic, your instructor, and your university do not control. It may change its content or behaviour, or be temporarily unavailable, at any time without notice. If a described feature or response is not available when you attempt this assignment, follow the Troubleshooting instructions provided for that task rather than guessing or inventing a result.

Record only what you genuinely observe. A test that does not produce an interesting result is still a valid, gradable finding — report it honestly as "not vulnerable" or "inconclusive."

This guide walks you through each practical task in Part A. Work through Task 1 first; it tells you where the relevant pages are then Tasks 2 and 3, which involve controlled testing. Tasks 4 and 5 are analytical and reporting tasks that draw on what you observed earlier.

Task 1: Website Exploration

Objective

Become familiar with the structure of Altoro Mutual and locate the three functionalities (Sign In, Search, Feedback) you will investigate in later tasks.

Prerequisites

- A modern web browser (Chrome, Firefox, or Edge).
- No prior security knowledge is required for this task.

Procedure

1. Open https://demo.testfire.net/ in your browser.
2. Browse the top navigation bar and the main menu. Visit at least three sections (for example: a Personal or Small Business banking page, the "Inside Altoro Mutual" section, and the Online Banking Sign In page).
3. As you browse, note down any five distinct features or functional elements you encounter — for example, a navigation menu, a product information page, a login form, a search box, a feedback/contact form, or a footer link such as a privacy policy or security statement page.
4. Locate the Sign In link (login area), the Search function, and the Feedback link specifically, and note the page or URL each one is on.
5. For each of these three functionalities, write down why it is the kind of feature that commonly needs security testing (think about what each one does with the information you type into it).

Expected Observations

Normal / properly-handled input: Not applicable to this exploratory task , simply record what you genuinely find on the site at the time you visit it.

Potentially noteworthy input handling: Not applicable , this task has no "vulnerable vs not vulnerable" result, only accurate observation.

Screenshot Instructions

- The homepage.
- Each of the three sections you navigated into.
- The Sign In page, the Search box, and the Feedback form.

Analysis — Answer in Your Report

- List your five features with a one-line description of each.
- State clearly where Sign In, Search, and Feedback are located.
- Explain the security relevance of each of these three functionalities.

Security Recommendations

- Not applicable for this task — recommendations begin in Task 2.

Troubleshooting

If the site is slow to load or a particular page is temporarily unavailable, wait a few minutes and retry, or choose a different section of the site to satisfy the "five features" requirement and note in your report which page was unavailable.

Task 2: SQL Injection Investigation

Objective

Learn to recognize surface-level indicators of a SQL injection vulnerability in a login form, using a safe, observational, black-box test — without writing or needing any real exploit code.

Prerequisites

- Completion of Task 1 (you should know where the Sign In page is).
- A basic understanding that a login form typically checks a username and password against a database using a query such as SELECT * FROM users WHERE username = … AND password = …
- No programming is required for this test; you will only be typing short text strings into the existing login form.

Procedure

1. Go to the Sign In / Online Banking login page on demo.testfire.net.
2. Baseline test: enter a made-up username and password (for example, "test" and "test") and submit the form. Record the exact message the application shows you — this is your baseline for an ordinary failed login.
3. Single-quote probe (username): in the username field only, enter a single apostrophe character (') and put any short text in the password field, then submit. An apostrophe is a special character in SQL syntax, so this is the standard first diagnostic step: if the application builds its database query by directly joining your input into the query text, a stray apostrophe can disrupt that query. Record exactly what is displayed.

4. Single-quote probe (password): repeat the same single-quote test, this time in the password field only, with an ordinary value in the username field. Record the result separately.
5. Compare both results to your baseline from Step 2. Note any difference in the message, page layout, HTTP status, or presence of any technical/database-related wording (for example, terms like "SQL", "exception", "driver", a stack trace, or an HTTP 500 error).
6. Conceptual illustration (optional, observe only): security textbooks commonly use the example input ' OR '1'='1' -- in a username field to illustrate, conceptually, how a flawed query could be made to always evaluate as true. If you choose to try this as an observational test, only record what the application actually displays. If you unexpectedly appear to be signed in, do not browse further into any account area — immediately stop, take your screenshot of that result only, and close the browser tab. Report the observation itself, not any further exploration.
7. Record all of your observations (baseline, username probe, password probe, and the optional conceptual test if attempted) in the Sample Analysis table in Part D.

Expected Observations

Normal / properly-handled input: All tests return the same generic message as your baseline (for example, a plain "invalid username or password" message), with no database-related wording and no change in page structure. This suggests the special character was handled safely.

Potentially noteworthy input handling: A test produces a response that is different from your baseline — for example, a database error, a stack trace, an HTTP 500 error, or any unexpected change in the page. This is a classic surface-level indicator that user input may be reaching the underlying query without safe handling. On its own, it does not prove a complete, working attack — record it as a noteworthy finding for further (instructor-guided) review, not as a confirmed breach.

Screenshot Instructions

- The baseline failed-login result.
- The result of the single-quote test in the username field.
- The result of the single-quote test in the password field.
- The result of the optional conceptual test, if you attempted it.

Analysis — Answer in Your Report

- Did any test produce a result different from your baseline? Describe the exact difference.
- Based only on what you directly observed, would you classify this functionality as Vulnerable, Not Vulnerable, or Inconclusive? Justify your answer using your own evidence.
- What would a real attacker potentially be able to do if this type of weakness existed in a production system?

Security Recommendations

- Use parameterized queries / prepared statements so user input is always treated as data, never as part of the query syntax.
- Apply the principle of least privilege to the database account the application uses.
- Return generic error messages to users and avoid exposing database or stack-trace details.

- Validate and constrain input formats as a defense-in-depth measure (not a substitute for parameterized queries).

Troubleshooting

Public demonstration sites are sometimes updated, reconfigured, or temporarily unreachable. If demo.testfire.net is unavailable, or your tests show no observable difference at all regardless of input, record this honestly as an "inconclusive — no observable difference" finding, and complete the equivalent exercise using PortSwigger Web Security Academy's free, officially provided "SQL injection" learning path (the introductory lab on SQL injection in a login/WHERE-clause context is the closest equivalent). Clearly state in your report which environment you actually used for this task.

Task 3: Cross-Site Scripting (XSS) Investigation

Objective

Learn to recognize surface-level indicators of an XSS vulnerability by observing how a web application handles and redisplays user-supplied text.

Prerequisites

- Completion of Task 1.
- A basic understanding that web pages are built from HTML, and that characters such as < and > have special meaning in HTML.

Procedure

1. Choose one input point to test: either the Search function or the Feedback form (use whichever one you confirmed in Task 1 actually displays your submitted text back to you). State which one you chose and why.
2. Baseline test: enter an ordinary, harmless phrase (for example, "test search" or "This is a test message") and submit. Record exactly how your text is displayed in the result.
3. HTML-marker test: enter a short, harmless string that includes HTML tags, such as testingboldtesting, and submit. Record whether the word "bold" is rendered in actual bold formatting (meaning your input was interpreted as HTML) or appears as plain visible text, such as bold (meaning it was safely encoded).
4. Standard alert() test: enter the well-known, harmless test string <script>alert('XSS-Test')</script> and submit. This exact string is used throughout security training, including official vendor and PortSwigger Web Security Academy materials, specifically because it only opens a harmless on-page pop-up box in your own browser — it does not affect the server, any other user, or any real data. Record whether a pop-up dialog appears (your input executed as script) or whether the text appears as plain, inert text (your input was safely encoded).
5. If a pop-up appears, take your screenshot, close the dialog, and stop. Do not modify the payload further or attempt to use it for anything beyond this observation.
6. Record all three observations in the Sample Analysis table in Part D.

Expected Observations

Normal / properly-handled input: Your text — including the < and > characters — appears on the page as plain, literal text exactly as typed, or is shown converted to safe equivalents (for example <script>), and no bold formatting or pop-up occurs. This indicates proper output encoding.

Potentially noteworthy input handling: The HTML-marker test actually renders in bold, and/or the alert() test produces a pop-up dialog. This shows the application is inserting your input into the page without encoding it, which is the underlying mechanism behind an XSS vulnerability.

Screenshot Instructions

- The baseline test result.
- The HTML-marker test result.
- The alert() test result (the pop-up, or the plain/encoded text if no pop-up occurred).

Analysis — Answer in Your Report

- Was your input reflected immediately on the same or next page (suggesting reflected XSS)? Note: confirming stored XSS would require checking whether the input reappears later or in a different browser session — if you did not verify this, report your finding as reflected/inconclusive rather than assuming it is stored.
- Which test(s), if any, indicated that your input was actively interpreted rather than safely encoded?
- Based only on your own evidence, classify the field as Vulnerable, Not Vulnerable, or Inconclusive, and justify your answer.

Security Recommendations

- Apply context-appropriate output encoding (for example, HTML-entity encoding) whenever user input is inserted into a page.
- Use a Content-Security-Policy (CSP) header to restrict what scripts are allowed to run.
- Use frameworks or templating engines that escape output by default.
- Validate input on the server side, and as a defense-in-depth measure, set the HttpOnly flag on sensitive cookies.

Troubleshooting

If demo.testfire.net does not reflect your input the way described, or the Search or Feedback forms are unavailable, record this honestly and complete the equivalent exercise using PortSwigger Web Security Academy's free "Cross-site scripting" learning path (the introductory "Reflected XSS into HTML context with nothing encoded" lab is the closest equivalent). Clearly state in your report which environment you actually used.

Task 4: Security Analysis

Objective

Apply the concepts from Tasks 2 and 3, together with general web-application and banking-security principles, to answer five analytical questions. This task does not require any further testing.

Approach

- Answer each question in your own words, using correct technical terminology.
- Where relevant, refer back to your own Task 2 and Task 3 observations to support your answers (for example, when discussing the CIA triad, connect it to what you actually recorded in Task 2).
- No new screenshots are required for this task.

Task 5: Report and Recommendations

Objective

Compile everything from Tasks 1 to 4 into a single, well-organized, professional security assessment report, submitted as a PDF with screenshots as stated on the cover page.

Guidance for Each Required Section

- **Introduction and objectives:** briefly state the purpose of the assessment and what you set out to evaluate.
- **Website overview:** summarize what Altoro Mutual is and the features you identified in Task 1.
- **Testing methodology:** describe the approach you followed for Tasks 2 and 3, including your baseline tests.
- **Tools used:** list your browser and any other tools (for example, browser developer tools) — most of this assignment needs nothing beyond a standard browser.
- **Screenshots and observations:** include every screenshot listed in Part C, each with a short caption explaining what it shows.
- **Findings and their potential impact:** use your completed Sample Analysis table from Part D as the core of this section.
- **Recommended security fixes:** map each recommendation directly to a specific finding — avoid generic, unconnected advice.
- **Conclusion:** summarize what you learned about web application security from this exercise.

Tips

- Keep a professional, objective tone throughout — write as a security assessor would, not as a story.
- Report inconclusive or negative results honestly; this is expected and still gradable.
- Build your report as you go — add screenshots and notes immediately after each test rather than trying to reconstruct everything at the end.
- Proofread before exporting, and submit as a PDF as stated on the cover page.

Part C — Screenshot Requirements

Your report must include genuine screenshots, taken by you, for each of the following. Each screenshot must be accompanied by a short explanation of what it shows.

- The Altoro Mutual homepage.
- The specific functionality under investigation for each task (Sign In page, Search box, Feedback form).
- The testing process itself, showing the input you entered.
- The observed response or output for each test, including baseline tests.
- Any confirmed vulnerability evidence, where applicable.
- Your final analysis or recommended fix for each finding.

Academic Integrity

Screenshots must be genuine and taken by you while completing this assignment. Do not copy images from the internet, from classmates, or from previous semesters.

Do not fabricate findings or claim a test succeeded when it did not. An honest "not vulnerable" or "inconclusive" result, properly documented, is a valid and gradable outcome.

Part D — Sample Analysis Format

Use the table below (or recreate it in your report) to record every test you perform across Tasks 1 to 3. The first row is a worked example showing the expected level of detail — replace it with your own genuine results; do not copy it.

Test ID	Feature Tested	Security Test	Observation	Evidence	Potential Impact	Recommended Fix	Result
EX-1	*Online Banking Sign In — username field*	*Baseline: entered a made-up username/password, then re-tested with a single quote (') in the username field*	*Example only — replace with your own genuine observation of what the application actually displayed*	*screenshot_02.png*	*Example: potential unauthorized access to account data if confirmed*	*Example: parameterized queries / prepared statements*	*Example — record your own result*
1							
2							
3							
4							
5							

Important: a test that does not produce a noteworthy result does not prove that the application is secure. Record such results as "Not Vulnerable" or "Inconclusive" with the reasoning behind that classification — this is a valid, gradable outcome, not a failure to find something.

Part E — Marking Rubric

Component	Marks	Full Marks	Partial Marks	Zero Marks
Website Exploration	3	All five features clearly identified and accurately described; all three functionalities correctly located; clear, specific security-relevance explanation for each.	Some features/functionalities identified but descriptions are vague or incomplete; security-relevance explanation present but shallow.	Feature list missing or incorrect; functionalities not identified; no security-relevance explanation.
SQL Injection Investigation	5	Correct conceptual explanation; correct identification of the underlying coding mistake; appropriate target justified; procedure followed with a genuine baseline and clearly recorded, correctly interpreted observations; CIA impact explained correctly; at least two valid prevention techniques.	Explanation mostly correct with minor errors; procedure attempted with gaps (e.g. missing baseline); interpretation present but weak; only one prevention technique or explained superficially.	Explanation absent or fundamentally incorrect; no genuine test evidence, or results appear fabricated/copied; no valid prevention techniques.
XSS Investigation	4	Correct reflected/stored distinction; appropriate field chosen and justified; procedure followed with genuine, correctly interpreted observations; impact explained correctly; valid prevention techniques given.	Partial or confused reflected/stored distinction; procedure attempted with gaps; interpretation present but weak; only one prevention technique.	Definitions missing or incorrect; no genuine test evidence; no valid prevention techniques.
Security Analysis	4	All five questions answered correctly and precisely, in the student's own words, with correct technical terminology.	Most questions correct but with gaps in precision or terminology; one or two answers incomplete.	Most or all answers missing, incorrect, or copied without understanding.
Report Quality & Screenshot Evidence	4	All required sections present and well organized; all required genuine screenshots included, labeled, and explained; findings table consistent with screenshots and narrative; inconclusive results reported honestly.	Report mostly complete but missing one or two sections or screenshots; some inconsistency between narrative and evidence; formatting issues.	Report substantially incomplete; screenshots missing, fabricated, or inconsistent with claimed findings.

Academic Integrity Note

Any submission found to contain fabricated screenshots, fabricated results, or evidence of testing an unauthorized target (any site other than demo.testfire.net or the named PortSwigger Web Security Academy labs) will be referred to the applicable academic integrity process, independent of the marks above.

Instructor: Hafiz Usama Ishtiaq
Faculty of Information Technology
University of Central Punjab
Lahore, Pakistan



Instructor: Hafiz Usama Ishtiaq
Faculty of Information Technology
University of Central Punjab
Lahore, Pakistan